

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.

PROJECT REPORT

On

CAMPUSSYNC: CAMPUS EVENT MANAGEMENT MOBILE APP (23CS4AEMAD)

Submitted by

TANISHK KAR (1BF24CS015)

TATAVARTHI CHIRANJEEVI SRIRAM (1BF24CS016)

AASTHA SHARMA K M (1BM24CS003)

ADITI KAMATH A (1BM24CS016)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Project work entitled “CAMPUSSYNC: CAMPUS EVENT MANAGEMENT MOBILE APP” carried out by **TANISHK KAR (1BF24CS015)**, **TATAVARTHI CHIRANJEEVI SRIRAM (1BF24CS016)**, **AASTHA SHARMA K M (1BM24CS003)**, and **ADITI KAMATH A (1BM24CS016)**, who are bonafide students of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The project report has been approved as it satisfies the academic requirements in respect of Mobile Application Development - (23CS4AEMAD) work prescribed for the said degree.

Ananya	Dr. Kavitha Sooda
Project In-charge Department of CSE BMSCE, Bengaluru	Professor and Head Department of CSE BMSCE, Bengaluru

ACKNOWLEDGEMENT

We express our sincere gratitude to B. M. S. College of Engineering and the Department of Computer Science and Engineering for providing the academic environment and resources required to complete this project. We are thankful to our project in-charge, Ananya, for her guidance and encouragement throughout the development of CampusSync. We also thank our Head of Department, faculty members, and classmates for their support, feedback, and suggestions that helped improve the usability and presentation of this mobile application.

TANISHK KAR (1BF24CS015)
TATAVARTHI CHIRANJEEVI SRIRAM (1BF24CS016)
AASTHA SHARMA K M (1BM24CS003)
ADITI KAMATH A (1BM24CS016)

ABSTRACT

CampusSync is a mobile application developed for BMS College of Engineering to simplify campus event discovery, registration, and administration. The application allows students to create an account, browse events from multiple clubs, search and filter events, register or unregister, complete a UPI-style payment flow for paid events, receive notifications, and view their personal event list. Club administrators can log in using pre-registered accounts, create events for their assigned club, view hosted events, inspect registration statistics, and export participant data.

The project is implemented using React Native and Expo with JavaScript. It uses React Navigation for screen flow, Context API for authentication state, AsyncStorage for persistence, Axios for API integration, and a complete mock-data fallback so that the application can be demonstrated without a live backend. The flagship feature is a real-time venue conflict detection mechanism that checks same-day bookings at the same venue and enforces a one-hour buffer between events. This report describes the objectives, architecture, requirements, implementation, testing, limitations, and future scope of CampusSync.

Index Sheet

Sl. No.	Chapter Title	Page No.
1	Introduction	6
2	Problem Statement	6
3	Objectives	6
4	Existing System and Proposed System	7
5	Requirement Analysis	7
6	Technology Stack	8
7	System Architecture	8
8	Module Description	9
9	Database and Data Model Design	10
10	Implementation Details	10
11	Venue Conflict Detection Algorithm	11
12	User Interface Design	12
13	Testing and Validation	13
14	Results and Discussion	13
15	Limitations and Future Scope	13
16	Conclusion and References	14
17	Appendix A: Suggested Demonstration Flow	15

Course outcomes:

CO	How CampusSync addresses the outcome
CO1	Applies mobile UI/UX design principles through a dark-mode design system, reusable cards, calendar view, feedback states, and role-based navigation.
CO2	Implements mobile app development concepts using React Native, Expo, navigation stacks, state management, persistence, and REST-ready API integration.
CO3	Demonstrates project communication through a working application, presentation deck, and formal project report.

1. Introduction

CampusSync is a campus event management mobile application designed for BMS College of Engineering. It brings event discovery, registration, scheduling, notifications, payments, and administrative management into one mobile-first platform. The application focuses on both student convenience and club administrator control, making it suitable for academic demonstrations and practical campus use.

The application supports six campus clubs: IEEE BMSCE, AI/ML Club, Cultural Club, Sports Committee, Photography Club, and Literary Club. Each club has sample events and two administrator accounts, enabling complete demonstration of role-based workflows. Students can browse events, apply filters, register for events, and view notifications. Admins can create events, manage hosted events, and view registration statistics.

The project is built with React Native and Expo, allowing it to run on Android, iOS, and web-supported development targets. It also includes an API layer that first attempts backend requests and then falls back to in-memory mock data and AsyncStorage persistence. This makes the project demonstration-ready even when the backend server is unavailable.

2. Problem Statement

In a college environment, event information is often shared through posters, class groups, social media announcements, or informal communication. This makes it difficult for students to discover all relevant opportunities in one place. Event organizers also face difficulty tracking registrations, managing capacity, sending reminders, and preventing venue clashes.

Campus clubs frequently use shared venues such as seminar halls, auditoriums, laboratories, and sports facilities. Without an automated conflict-checking system, two clubs may attempt to book the same venue at overlapping times, or schedule back-to-back events without sufficient setup and cleanup time. This creates operational confusion and poor student experience.

CampusSync solves this problem by providing a centralized mobile application where students and administrators can interact with the same event ecosystem while following role-based permissions and automated scheduling rules.

3. Objectives

- To develop a mobile application for discovering and managing campus events.
- To provide separate workflows for students and club administrators.
- To allow students to register, unregister, view payment flow, and track their events.
- To allow administrators to create events only for their assigned club.
- To implement real-time venue conflict detection with a mandatory one-hour buffer.
- To provide admin statistics, registered-student lists, and CSV export capability.
- To support offline project demonstrations through mock data and AsyncStorage persistence.

- To design a polished dark-mode user interface with reusable components and clear navigation.

4. Existing System and Proposed System

Existing System

The existing event management approach in many colleges is mostly manual or fragmented. Students learn about events through posters, WhatsApp groups, emails, or word of mouth. Registration may be handled through separate forms for each event. Administrators often maintain participant lists manually and check venue availability through informal coordination.

Limitations of Existing System

- No single platform for all campus events.
- Manual registration tracking and limited participation analytics.
- Higher chance of duplicate or conflicting venue bookings.
- No unified notification center for reminders and confirmations.
- Difficult student discovery across different clubs and categories.

Proposed System

The proposed system, CampusSync, is a mobile application that centralizes event discovery, registration, payment demonstration, notifications, and admin management. It enforces club ownership rules for administrators and checks venue conflicts before event creation. The app is also backend-ready while remaining fully demonstrable offline through mock data.

5. Requirement Analysis

Functional Requirements

Requirement	Description
User registration	Students can create accounts using a valid BMSCE email and academic details.
Login	Students and pre-registered admins can authenticate with role validation.
Event discovery	Users can view events, featured cards, filters, search results, and calendar dots.
Registration	Students can register or unregister for events and view capacity status.
Payment flow	Paid events show a UPI QR-style screen, timer, and receipt confirmation.
Admin event creation	Admins can create events only for their own club.
Conflict detection	The app checks venue, date, start time, end time, and one-hour buffer rules.
Statistics	Admins can view club-level registration data and participant lists.
Notifications	Users can view reminders, confirmations, and new event messages.

Non-Functional Requirements

- Usability: interface should be simple, mobile-friendly, and role-aware.
- Performance: event lists and filters should respond quickly for mock data size.
- Reliability: mock fallback should keep the app functional if the backend is unavailable.
- Maintainability: screens, services, context, navigation, and theme files should remain modular.
- Security: admin-only actions should be guarded by role and club ownership checks.

Hardware and Software Requirements

Category	Requirement
Hardware	Laptop or desktop with Node.js development environment; Android device/emulator or Expo Go supported phone.
Operating system	Windows, macOS, or Linux development environment.
Runtime	Node.js 18 or later.
Mobile framework	Expo SDK 54 with React Native 0.81.5.
Editor	Visual Studio Code or equivalent IDE.
Optional backend	Node.js/Express API and MongoDB, if live backend mode is required.

6. Technology Stack

Layer	Technology	Purpose
Framework	React Native with Expo SDK 54	Cross-platform mobile application development.
Language	JavaScript ES2022+	Screen logic, API handling, validation, and UI code.
Navigation	React Navigation native stack and bottom tabs	Authentication flow, tab navigation, and detail screens.
State	React Context API	Global authentication state and user session management.
Persistence	AsyncStorage	Local storage of token and registered users.
API client	Axios	HTTP calls to backend endpoints with mock fallback.
UI support	Expo Linear Gradient and Icons	Visual design, buttons, icons, and tab bar.
Animation	React Native Reanimated	Smooth UI feedback and interaction effects.

7. System Architecture

CampusSync is organized as a client-side mobile application with a backend-ready API layer. The architecture separates entry configuration, authentication state, navigation, screens, reusable UI

components, services, and theme tokens.

Architecture Flow

User opens app -> AuthProvider loads saved session -> RootNavigator chooses authentication stack or main tabs -> User performs student/admin action -> API layer attempts backend request -> Mock fallback returns local data if backend is unavailable -> AsyncStorage persists relevant state.

Project Structure

Path	Responsibility
App.js	Root entry point, providers, navigation container, and global dark theme.
src/context/AuthContext.js	Authentication state, login/logout/register handling, and persistence.
src/navigation/RootNavigator.js	Auth stack, main bottom tabs, and nested event/admin navigation.
src/services/api.js	Clubs, admins, mock events, authentication APIs, event APIs, notifications, and profile APIs.
src/components	Reusable EventCard, CalendarView, and EmptyState components.
src/screens	Authentication, home, event detail, payment, admin, notification, my events, and profile screens.
src/theme/index.js	Color, spacing, typography, and design tokens.

8. Module Description

Authentication Module

The authentication module supports two roles: student and administrator. Students register using personal details and then log in. Admins use predefined accounts mapped to clubs. The app validates whether a selected role matches the email type and prevents admin accounts from being used for student registration.

Home and Discovery Module

The Home screen displays upcoming events, featured events, personalized stats, category filters, club filters, search with autocomplete, recent searches, and a toggleable calendar view. Events can be filtered by title, description, tags, and club.

Event Detail and Registration Module

The event detail screen displays event metadata including description, organizer, date, time, venue, capacity, price, registration deadline, and activity points. Students can register or unregister. Paid events navigate to the payment screen.

Payment Module

The payment module demonstrates a UPI QR-style payment workflow. It includes a deterministic QR-like visual, five-minute timer, and digital receipt behavior. This supports project demonstration without connecting to a live payment gateway.

Admin Event Creation Module

Administrators can create events only for their assigned club. The create event form includes date, time, end time, venue, category, capacity, price, description, and activity points. Real-time venue conflict detection prevents invalid submissions.

Admin Statistics Module

The admin statistics module provides club-level summaries and per-event drill-downs. Admins can inspect registered students, search and sort lists, and export data as CSV for offline use.

Notification and Profile Modules

Notifications show registration confirmations, reminders, and new event messages. The Profile screen displays user information, role/branch metadata, app information, help, privacy, terms, and sign-out functionality.

9. Database and Data Model Design

The current mobile repository uses in-memory mock data and AsyncStorage persistence for demonstration. The same structures map naturally to backend collections or database tables if a Node.js/Express and MongoDB backend is connected.

Entity	Important Fields
Club	id, name, icon, color
Admin	email, clubId, name, password for demo authentication
Event	_id, title, description, category, club, clubId, activityPoints, date, time, endTime, location, price, organizer, maxCapacity, registeredCount, registeredStudents, tags, imageUrl, registrationDeadline, createdBy
Notification	_id, title, body, type, read, createdAt, eventId
Registered User	email, password, name, username, branch, semester, gender, registeredAt

Sample Data Volume

- 6 supported clubs.
- 12 pre-registered administrator accounts.
- 18 mock events, with 3 sample events per club.
- Generated registered-student lists for admin statistics demonstration.

10. Implementation Details

Authentication Implementation

The API layer first attempts to authenticate using a backend endpoint. If the backend is unavailable, it falls back to local validation. Admin credentials are matched against predefined admin records. Student credentials are loaded from AsyncStorage and checked locally. The login

response returns a mock token and a user object containing role, branch, club, hosted events, and registered events.

Event API Implementation

Event fetching supports filters for category, club, search, and sorting. Search checks the event title, description, tags, and club name. Event creation and deletion are restricted by admin role and club ownership. Registration and unregistration update registered counts and student lists in the mock event data.

Offline Fallback

Every main API method attempts a network request using Axios. If the request fails, the method returns locally available mock data. This design allows the app to behave like a backend-connected application while still being reliable during classroom demonstrations.

Authorization Rules

- Only admin emails can log in with the admin role.
- Admin emails cannot be used for student self-registration.
- Admin users can create events only for their assigned club.
- Admin users can delete only events that belong to their own club.
- Students can register for events but cannot access admin-only operations.

11. Venue Conflict Detection Algorithm

The venue conflict detection algorithm is the most important scheduling feature in CampusSync. It prevents two clubs from booking the same venue at overlapping times and also enforces a one-hour gap before and after existing events. This buffer is useful for setup, cleanup, audience movement, and technical preparation.

Algorithm Steps

- 1 Accept location, date, start time, end time, and optional excluded event ID as input.
- 2 Return no conflict if location, date, or start time is missing.
- 3 Convert the requested event date into a normalized date key.
- 4 Parse the requested start and end times into minutes from midnight.
- 5 For each existing event, compare venue names after trimming and converting to lowercase.
- 6 Check whether the existing event is on the same date.
- 7 Parse the existing event start and end times.
- 8 Expand the existing event block by 60 minutes before and after.
- 9 Use interval overlap logic to decide if the requested event overlaps the buffered block.

- 10 Return conflict details including title, club, date, time, end time, and location.

Overlap Condition

A conflict exists when both of the following conditions are true:

```
requestedStart < otherEnd + buffer
otherStart - buffer < requestedEnd
```

If the conditions are satisfied, the submit button is disabled and the admin is shown a warning banner with the conflicting event details.

12. User Interface Design

CampusSync uses a dark-mode design system based on consistent colors, card styles, gradients, and iconography. The UI is designed for quick scanning of event information on mobile screens.

Token	Value	Purpose
Background	#080818	Deep dark blue app background.
Card background	rgba(255,255,255,0.05)	Panels and event cards.
Primary	#8B5CF6	Main actions and active navigation.
Secondary	#3B82F6	Supporting blue accent.
Accent	#10B981	Success state and positive indicators.
Warning	#F59E0B	Timers and caution messages.
Danger	#EF4444	Errors and destructive actions.

Main Screens

Screen	Main Purpose
Welcome	Entry point with Sign Up and Login routing.
Login	Role selector, validation, admin club logic, and error feedback.
Register	Student signup with password strength and academic details.
Home	Event discovery, filters, search, calendar, and reminders.
Event Detail	Complete event information and registration call-to-action.
Create Event	Admin-only event creation with conflict checking.
Payment	UPI-style QR payment timer and receipt.
My Events	Registered, hosting, and archived event tabs.
Admin Stats	Club statistics and event registration drill-down.
Notifications	Confirmation, reminder, and new event messages.
Profile	User information, settings, modals, and sign out.

13. Testing and Validation

The project can be validated through manual functional testing using Expo Go, an Android emulator, or the web target. The following test cases cover the core workflows.

Test Case	Expected Result
Student registration with valid BMSCE email	Account is created and user is asked to log in.
Student login after registration	Main tab interface opens with student permissions.
Admin login using admin email	Admin user opens with club ownership metadata.
Admin email with student role	Login is rejected with role mismatch message.
Search for event keyword	Matching events are shown.
Filter by club/category	Only matching events remain visible.
Register for free event	Registered count and My Events data are updated.
Open paid event	Payment screen appears before confirmation flow.
Create valid admin event	Event is added to the hosted club event list.
Create conflicting venue booking	Submit is blocked and conflict warning appears.
Delete other club event as admin	Action is rejected by authorization rule.
Mark notification as read	Notification read status changes.

14. Results and Discussion

CampusSync successfully demonstrates a complete mobile event-management workflow. Students can discover events from all supported clubs, register for activities, view personalized event lists, and receive notifications. Administrators can create and manage events within club boundaries and inspect participant information.

The room conflict detection feature is particularly useful because it converts manual venue coordination into a predictable validation rule. By checking same venue, same date, time range, and one-hour buffer, the application reduces the chance of operational clashes between clubs.

The mock fallback strategy also improves demonstration reliability. Even without a running backend, the application retains realistic behavior for authentication, event listing, filtering, registration, notification, and admin statistics.

15. Limitations and Future Scope

Limitations

- The backend server is optional and not included in this repository.
- Payment processing is a project demonstration and not connected to a real payment gateway.
- Admin demo credentials are stored in the mobile source and should be moved server-side for production.

- Some mock event changes are in-memory and may reset when the app process restarts.
- Production deployment would require hardened authentication, secure storage, audit logs, and database-backed reporting.

Future Scope

- Full Node.js/Express backend integration with MongoDB.
- Secure password hashing, JWT refresh tokens, and server-side authorization.
- Real UPI deep-link or payment-gateway integration.
- Push notifications using Expo Notifications or Firebase Cloud Messaging.
- Faculty approval workflow for event publishing.
- QR-based event attendance tracking.
- Activity-point certificates and analytics dashboards.
- Waitlists for full events and automated reminder scheduling.

16. Conclusion

CampusSync provides a practical and well-scoped solution for campus event management. It combines student-facing discovery and registration features with administrator tools for event creation, statistics, and conflict-safe venue scheduling. The project demonstrates mobile app development concepts such as navigation, state management, persistence, API integration, reusable components, and responsive UI design.

By supporting offline mock data and a backend-ready API layer, CampusSync is suitable both for academic demonstration and future production expansion. The venue conflict detection algorithm adds meaningful real-world value by addressing a common coordination problem in college event planning.

17. References

- 1 React Native Documentation - <https://reactnative.dev/>
- 2 Expo Documentation - <https://docs.expo.dev/>
- 3 React Navigation Documentation - <https://reactnavigation.org/>
- 4 AsyncStorage Documentation - <https://react-native-async-storage.github.io/async-storage/>
- 5 Axios Documentation - <https://axios-http.com/>
- 6 CampusSync project source code and README documentation.

APPENDIX A: Suggested Demonstration Flow

- 1 Launch the app and show the Welcome screen.
- 2 Register a new student account using a BMSCE email.
- 3 Log in as the student and browse the Home screen.
- 4 Search for an event and apply filters.
- 5 Open Event Detail and register for an event.
- 6 Open a paid event and demonstrate the UPI-style payment flow.
- 7 View My Events and Notifications.
- 8 Sign out and log in as a club administrator.
- 9 Create a valid event for the admin's club.
- 10 Attempt to create a conflicting room booking and show the warning.
- 11 Open admin statistics and participant-list export flow.